

Bản trình chiếu: Skip list và ứng dụng

Wei Ran

2005

Nguồn gốc: Làm cho hiệu suất thuật toán “nhảy lên”: bản trình chiếu về skip list

IOI China candidate team papers / 2005 / Wei Ran / Wei Ran.ppt

1 Mạch chính của slide

Bản trình chiếu đi cùng bài viết về skip list. Các mốc đọc được gồm *Skip List*, năm 1987, các độ phức tạp $O(\log n)$, $O(n)$, hàm `random()`, và các bài *Cashier* của NOI 2004 Day 1, HNOI 2004 Day 1. Bản ghi chú này dựa trên bài viết đã dịch để trình bày lại luồng slide: cấu trúc của skip list, ba thao tác cơ bản, phân tích xác suất, rồi hai ứng dụng thi đấu.

2 Cấu trúc

Skip list là danh sách liên kết nhiều tầng. Tầng dưới cùng chứa mọi phần tử theo thứ tự, còn các tầng cao hơn chứa một tập con được chọn ngẫu nhiên. Khi tìm kiếm, ta đi ngang ở tầng cao cho đến khi không thể đi tiếp, rồi hạ tầng.

Một skip list có các danh sách

$$S_0, S_1, \dots, S_h.$$

Mỗi danh sách đều có hai nút biên $-\infty$ và $+\infty$. Danh sách S_0 chứa toàn bộ khóa theo thứ tự tăng dần; với $i > 0$, các phần tử ở S_i là một tập con của S_{i-1} . Vì vậy một khóa xuất hiện ở tầng cao cũng xuất hiện ở mọi tầng thấp hơn, tạo thành một cột.

Cách nhìn trực quan là: tầng thấp cho độ chính xác, tầng cao cho bước nhảy dài. Cây cân bằng giữ chiều cao bằng các phép quay; skip list giữ chiều cao tốt bằng ngẫu nhiên hóa. Đây là lý do nó thường dễ cài hơn AVL tree hoặc red-black tree nhưng vẫn đạt hiệu suất gần tương đương trong thực nghiệm.

3 Tìm kiếm, chèn và xóa

Tìm kiếm khóa x bắt đầu từ đầu danh sách ở tầng cao nhất. Tại vị trí hiện tại, nhìn sang nút bên phải có khóa y :

- nếu $x = y$, tìm kiếm thành công;
- nếu $x > y$, đi sang phải;
- nếu $x < y$, đi xuống một tầng.

Ba nhánh $x = y$, $x > y$, $x < y$ là các mốc còn đọc được trực tiếp từ slide. Nếu đã xuống dưới tầng đáy mà chưa gặp x , khóa không tồn tại.

Chèn một khóa gồm hai quyết định: vị trí và chiều cao cột. Vị trí được tìm bằng đường tìm kiếm chuẩn, đồng thời lưu lại trên mỗi tầng nút đứng ngay trước vị trí chèn. Chiều cao được quyết định bằng mô-đun ngẫu nhiên mà slide ghi dưới dạng:

$$r \leftarrow \text{random}(), \quad \begin{cases} \text{làm A,} & r < p, \\ \text{làm B,} & r \geq p. \end{cases}$$

Nói cách khác, bắt đầu với chiều cao 1; mỗi lần gieo thành công thì tăng thêm một tầng, gieo thất bại thì dừng. Khi đó

$$\Pr(\text{chiều cao} \geq k) = p^{k-1}.$$

Nếu cột mới cao hơn skip list hiện tại, ta thêm các tầng mới phía trên.

Xóa khóa x cũng dùng đường tìm kiếm để lấy các nút đứng trước cột chứa x , rồi bỏ cột này khỏi mọi tầng. Nếu các tầng cao nhất chỉ còn hai nút biên, chúng được xóa khỏi cấu trúc.

4 Độ phức tạp

Nhờ chọn tầng bằng ngẫu nhiên, số bước kỳ vọng cho tìm kiếm, chèn và xóa là $O(\log n)$. Cài đặt đơn giản hơn nhiều cây cân bằng, nhưng vẫn hỗ trợ tốt các thao tác thứ tự nếu lưu thêm kích thước đoạn.

Phân tích bộ nhớ dựa trên tổng kỳ vọng số nút. Với n khóa, số cột xuất hiện ở tầng i xấp xỉ np^{i-1} . Tổng số nút kỳ vọng là

$$\sum_{i \geq 1} np^{i-1} = \frac{n}{1-p},$$

nên bộ nhớ là $\mathcal{O}(n)$ khi p là hằng số nhỏ hơn 1.

Phân tích tìm kiếm có thể nhìn ngược từ nút đích về góc trái trên. Ở một tầng, nếu cột hiện tại không vượt lên tầng trên, đường đi ngược phải đi sang trái; nếu cột vượt lên, đường đi ngược đi lên. Gọi $C(k)$ là số bước kỳ vọng để đi ngược qua k tầng, ta có hệ thức

$$C(k) = (1-p)(1 + C(k)) + p(1 + C(k-1)),$$

suy ra $C(k) = k/p$. Vì chiều cao kỳ vọng là $\mathcal{O}(\log n)$, thời gian tìm kiếm, chèn và xóa đều là $\mathcal{O}(\log n)$ theo kỳ vọng.

Slide cũng nhắc tới tìm kiếm có nhớ. Nếu hai truy vấn liên tiếp nằm gần nhau, ta có thể giữ lại các vị trí trên đường tìm kiếm trước đó và bắt đầu lần tìm kiếm sau từ một “ngón tay” gần mục tiêu. Khi khoảng cách thứ tự giữa hai khóa là k , thời gian kỳ vọng có thể giảm xuống $\mathcal{O}(\log k)$. Kỹ thuật này chỉ đáng dùng khi dữ liệu có tương quan; với truy vấn ngẫu nhiên, chi phí duy trì thông tin nhớ có thể không bù lại lợi ích.

5 Cashier

Các bài như *Cashier* cần duy trì tập số động, truy vấn thứ hạng hoặc xóa nhiều phần tử theo ngưỡng. Skip list là một lựa chọn tự nhiên khi cần cấu trúc cân bằng động nhưng muốn cài đặt gọn.

Trong *Cashier*, có bốn loại thao tác: thêm nhân viên với lương ban đầu, tăng toàn bộ lương, giảm toàn bộ lương, và hỏi lương lớn thứ k . Khi lương sau khi giảm thấp hơn ngưỡng hợp đồng, nhân viên rời công ty và phải bị xóa khỏi cấu trúc. Một biến bù toàn cục xử lý tăng giảm đồng loạt; cấu trúc dữ liệu chỉ lưu lương đã trừ bù.

Có ba lựa chọn tự nhiên:

segment tree theo miền lương	$I : \mathcal{O}(\log R)$	$S : \mathcal{O}(\log R)$	$F : \mathcal{O}(\log R)$
splay tree theo phần tử	$I : \mathcal{O}(\log N)$	$S : \mathcal{O}(\log N)$	$F : \mathcal{O}(\log N)$
skip list	$I : \mathcal{O}(\log N)$	$S : \mathcal{O}(\log N)$	$F : \mathcal{O}(\log N)$

Ở đề gốc, miền lương không quá lớn nên segment tree rất nhanh. Nhưng khi miền khóa lớn hoặc khóa không phải số nguyên nhỏ, cấu trúc theo miền trở nên kém linh hoạt. Skip list cạnh tranh trực tiếp với splay tree: cùng độ phức tạp kỳ vọng cho chèn, xóa và hỏi hạng, nhưng cài đặt ít thao tác cân bằng hơn.

Để hỗ trợ hỏi hạng, mỗi liên kết ngang có thể lưu số phần tử bị bỏ qua, hoặc mỗi nút lưu kích thước khoảng tương ứng. Khi đi tìm phần tử lớn thứ k , ta đi từ tầng cao xuống, giống tìm kiếm thường, nhưng quyết định đi ngang dựa trên số phần tử mà bước nhảy sẽ vượt qua.

6 *Pet Adoption*

Mốc *HNOI 2004 Day 1* trong slide tương ứng với bài nhận nuôi thú cưng. Mỗi sự kiện đưa đến một thú nuôi hoặc một người nhận nuôi với trị đặc trưng x , trong đó

$$0 < x < 2^{31}.$$

Nếu phía đối ứng đang chờ, ta ghép với trị gần x nhất; nếu hòa, chọn trị nhỏ hơn. Chi phí bất mãn cộng thêm là $|x - y|$, và số sự kiện tối đa đọc được từ bài viết là 80000.

Bài này cho thấy rõ ưu thế của skip list so với segment tree theo miền khóa. Miền 2^{31} quá rộng để dựng cây theo toàn bộ giá trị; nếu dùng cấp phát động thì cấu trúc vẫn phụ thuộc mạnh vào phân bố khóa. Skip list chỉ phụ thuộc vào số phần tử đang chờ. Để xử lý một sự kiện, tìm phần tử trước và sau x , chọn phần tử có khoảng cách nhỏ hơn, xóa nó nếu ghép thành công; nếu không có phía đối ứng, chèn x vào tập chờ hiện tại.

7 Kết luận

Thông điệp của slide là skip list không chỉ là một mẹo xác suất. Nó là một cấu trúc dữ liệu cân bằng theo nghĩa thực dụng: thao tác chính ngắn, phân tích kỳ vọng rõ, và mở rộng tốt cho các bài thứ tự động như truy vấn hạng, tìm tiền nhiệm–kế nhiệm, hoặc xóa theo ngưỡng. Khi bài toán không phụ thuộc vào một miền khóa nhỏ cố định, skip list là một lựa chọn đáng cân nhắc bên cạnh các cây cân bằng quen thuộc.